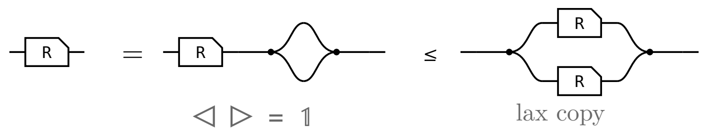
The allegory axioms: the proofs

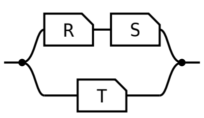
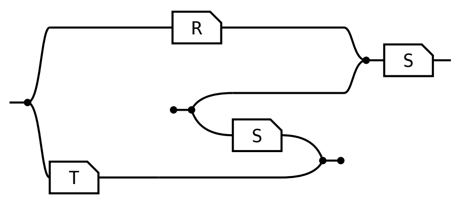
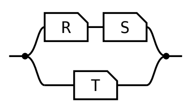
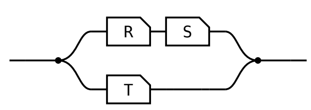
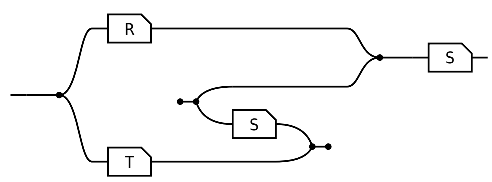
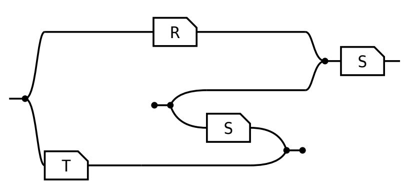
The companion to `allegory-axioms.typ`, which states these laws and their pictures. Here each one is worked: the steps of the Lean proof, side by side, with the rule that reaches each under it.

§1. `R ∩ R = R`, the one that is not bookkeeping

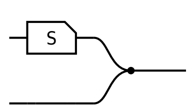
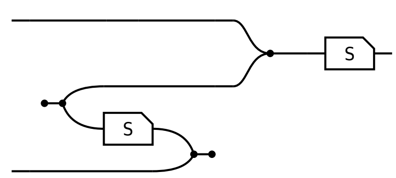
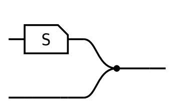


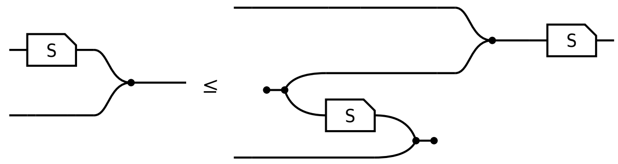
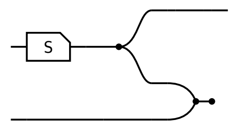
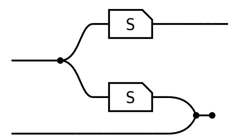
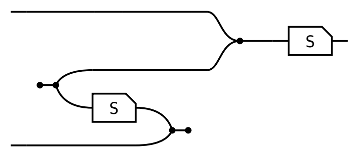
The last picture is `R ∩ R` itself, so this is `R ≤ R ∩ R` and the lax copy law is the whole of it. The other direction is not proved here: `R ∩ S ≤ R` holds for every `S`, by weakening `S` to `⊤` and then collapsing `R ∩ ⊤` — which is the paper's own remaining three steps, packaged once.

§2. The modular law, from Frobenius

the modular law, `R S ∩ T ≤ (R ∩ T S°) S`	 ≤ 	
term	**picture**	**the rule that reaches it**
`(R S) ∩ T`		**the start:** `R S ∩ T`.
`= (◁ (R ⊗ T)) ((S ⊗ 𝟙) ▷)`	= 	**Factor out the shared prefix:** `(R S) ⊗ T` is `(R ⊗ T) (S ⊗ 𝟙)`.
`≤ (◁ (R ⊗ T)) ((𝟙 ⊗ S°) (▷ S))`	≤ 	**The merge slide.** `S` slides through the merge and reappears as `S°` on the other strand. **The only inequality in the proof.**
`= (R ∩ (T S°)) S`	= 	**Fold the prefix back in;** the copy-merge pair reads as a meet again.

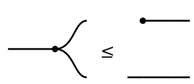
`S` occurs once on the left of the slide and twice on the right, and the lax copy law is the only law in the definition that may duplicate a box — so that is where the inequality has to be.

the merge slide, `(S ⊗ 𝟙) ▷ ≤ (𝟙 ⊗ S°) ▷ S`	 ≤ 	
term	**picture**	**the rule that reaches it**
`(S ⊗ 𝟙) ▷`		**the start:** `(S ⊗ 𝟙) ▷`.

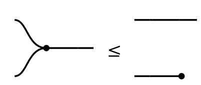
the merge slide, (S ⊗ 𝟙) ▷ ≤ (𝟙 ⊗ S°) ▷ S		
term	**picture**	**the rule that reaches it**
= ((S ◁) ⊗ 𝟙) (𝟙 ⊗ ▷⊸)	= 	**Rebuild the merge** from a copy and a right bracket, the shape the lax copy law applies to.
≤ ((◁ (S ⊗ S)) ⊗ 𝟙) (𝟙 ⊗ ▷⊸)	≤ 	**The lax copy law:** S ◁ becomes ◁ (S ⊗ S), the box copied. **The whole of the modular law, in one step.**
= (𝟙 ⊗ S°) (▷ S)	= 	**Reshape back,** and the surviving duplicate slides round the right bracket to become S°.

§3. R ≤ R R° R, by cutting two wires

Freyd reads this off the modular law (§2.112): R ⊑ 1R ∩ R ⊑ (1 ∩ R R°) R ⊑ R R° R. In pictures neither the modular law nor a meet with 1 is wanted. The one law that makes a picture bigger is spent twice, and it is the cut, 𝟙 ≤ ⊸ ⟜:

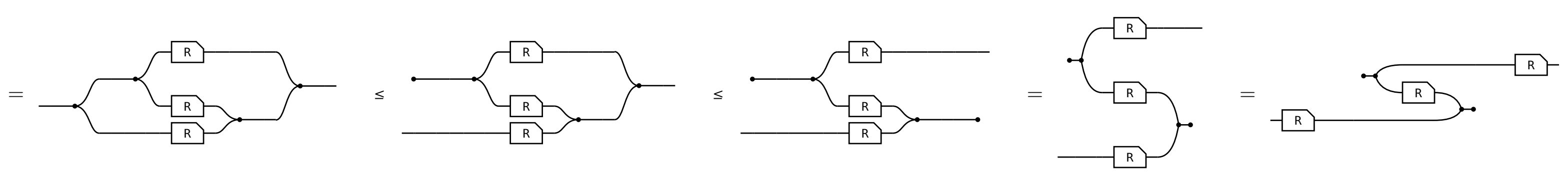


cut the copy's left output and what comes out there is **anything at all**



cut the merge's right input and it is **simply discarded**

R is its own three-fold meet — three copies of the box between a copy tree and a merge tree — and the two cuts turn the trees into a ⟜◁ and a ▷⊸. The three boxes never move:

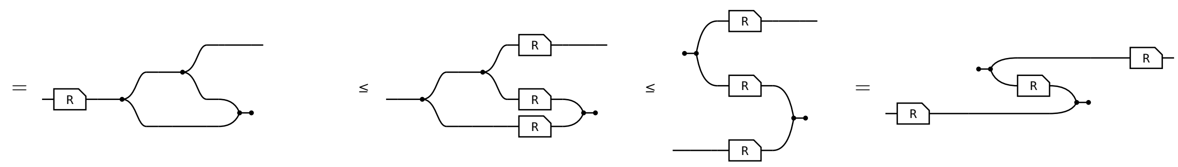


R ∩ R ∩ R = R — cut — cut — ⟜◁, ▷⊸ — that is R°

Read the last picture from the input, which is now the bottom strand: through the bottom box, up the ▷⊸ and back along the middle box — which is that box turned round, so it is R° — down the ⟜◁ and out along the top box. Where the meet had one input feeding all three boxes and one output draining all three, the zig-zag has both of those **cut**, and a cut wire is the one thing this calculus lets you add for free.

§4. The same, from the cap end

The section above cuts twice in its chain. This one cuts once and pays for the other cut with the **lax copy law** — `R ◁ ≤ ◁ (R ⊗ R)`, the only law in the definition that may duplicate a box. Read `𝟙` as a copy tree whose last two legs are capped back:



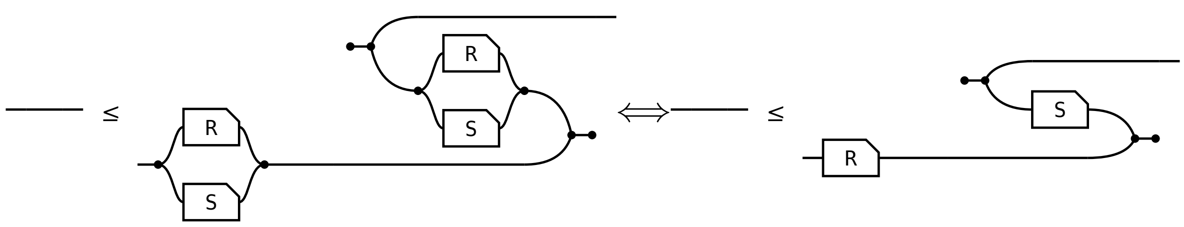
`◁ (◁ ⊗ 𝟙) (𝟙 ⊗ ▷⊸) = 𝟙` lax copy, twice cut that is `R°`

The three boxes are not there to begin with: there is one, and the two copy dots walk back **through** it, leaving it behind three times. So the `▷⊸` is the frame's own cap before anything is cut, and the copy tree is the only thing left to cut.

Neither chain is cheaper in what it **spends**. `◁ ▷ = 𝟙` is itself the merge cut, so the second `𝟙 ≤ ⊸ ⟜` is underneath this one; `R ∩ R = R` is the lax copy law, so that is underneath the section above. Both routes pay both. What differs is the **layer**: this chain names only the copy, the cap and the box, so it holds as soon as the converse does — before `∩` exists. The section above climbs into the meet semilattice and back down to state a containment in which no meet occurs.

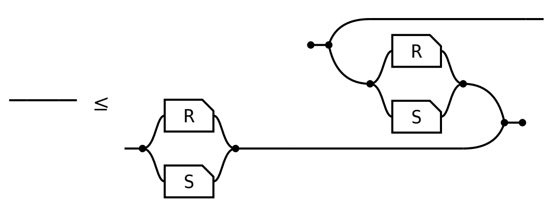
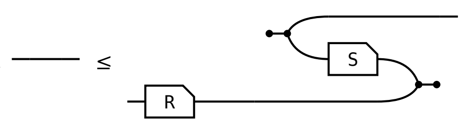
§5. Entireness of an intersection

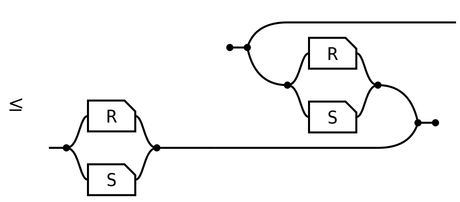
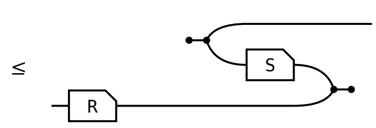
`R` is **entire** when `𝟙 ⊑ R R°`. When is an **intersection** entire?

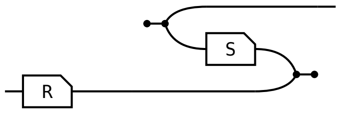
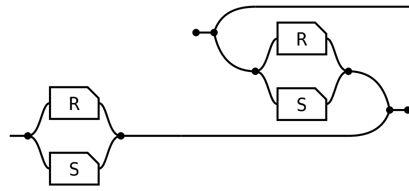


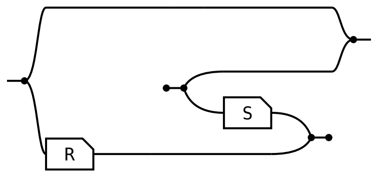
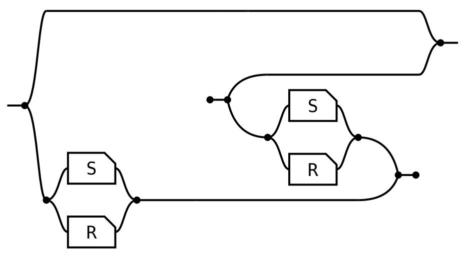
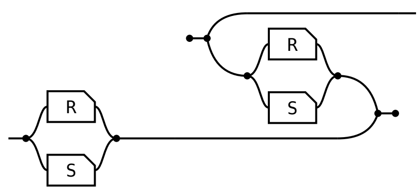
`Total (R ∩ S) ↔ 𝟙 ≤ R S°`

On the left `R ∩ S` is named **twice**; on the right `R` and `S` once each and the meet is gone. The meet is needed to **ask** whether something is entire, not to **answer** it. Where `⊑` is **defined** as `X ∩ Y = X` the two sides are one proposition; here `≤` is primitive, so both directions are real steps — and they cost very different things.

⟹ given `R ∩ S` entire,  show 

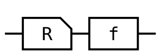
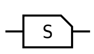
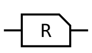
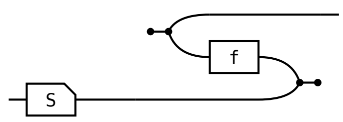
term	**picture**	**the rule that reaches it**
`𝟙`		**the start:** `𝟙`.
`≤ (R ∩ S) (R ∩ S)°`		**the hypothesis:** `𝟙 ≤ P P°` at `P = R ∩ S`.
`≤ R S°`		**monotonicity, twice.** No modular law.

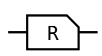
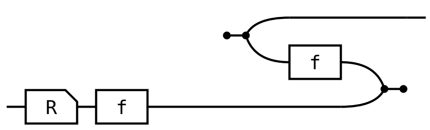
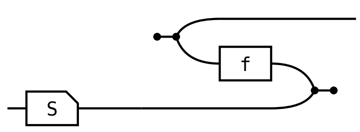
⟸ given —— ≤  show `R ∩ S` entire, —— ≤ 

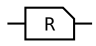
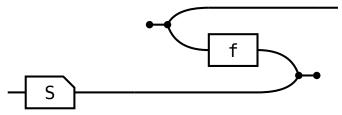
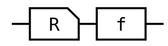
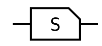
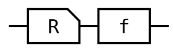
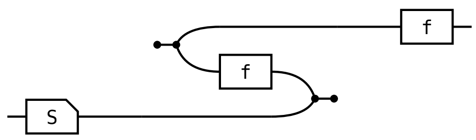
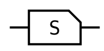
term	picture	the rule that reaches it
`𝟙`	——	**the start:** `𝟙`.
`≤ 𝟙 ∩ (R S°)`	≤ 	**the hypothesis,** met with `𝟙`.
`= 𝟙 ∩ ((S ∩ R) (S ∩ R)°)`	= 	`𝟙 ∩ R S° = 𝟙 ∩ (S ∩ R)(S ∩ R)°`. `R S°` relates `a` to `a′` when some `b` has `a R b` and `a′ S b`; the `𝟙 ∩` sets `a′ = a`, and it reads “some `b` has `a R b` and `a S b`” — one arrow of `R ∩ S`. Two arrows become one, and that is what needs the modular law.
`≤ (R ∩ S) (R ∩ S)°`	≤ 	**`X ∩ Y ≤ Y`,** then `S ∩ R = R ∩ S`.

§6. The shunting rule

A map moves from one side of a containment to the other at the cost of its converse, `R f ≤ S ⟺ R ≤ S f°`. It is how a function gets out of the way so the relation underneath can be reasoned about. Each direction spends exactly one half of **map** — never both.

shunting right, ⟹ given  ≤  show  ≤ 

term	picture	the rule that reaches it
`R`		**the start:** `R`.
`≤ R (f f°)`	≤ 	**total:** `𝟙 ≤ f f°`, so `f f°` may be inserted after `R`.
`≤ S f°`	≤ 	Re-bracket to `(R f) f°` and apply the hypothesis.

shunting right, ⟸ given  ≤  show  ≤ 		
term	**picture**	**the rule that reaches it**
`R f`		**the start:** `R f`.
`≤ S (f° f)`	≤ 	Apply the hypothesis on the left; the two `f`s are now adjacent.
`≤ S`	≤ 	**single valued:** `f° f ≤ 𝟙`, so the pair cancels.

Shunting left is the mirror, same shape, map on the other side throughout. What the rule is, is an adjunction: `f ⊣ f°`, with **total** the unit and **single valued** the counit, so “`f` is a map” and “`f` has a right adjoint, namely `f°`” are one statement.